

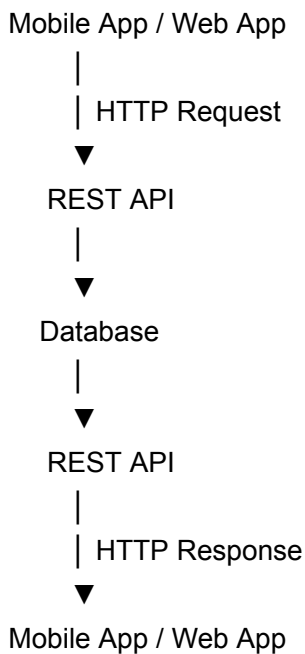
REST API Fundamentals

To build production-level APIs, first understand how a REST API communicates, how requests and responses are structured, how resources are designed, and how authentication, validation, errors, security, and scalability fit together.

1. What is HTTP?

HTTP (HyperText Transfer Protocol) is a communication protocol used between a client and a server.

For example:



HTTP defines:

- Request
 - Response
 - Methods
 - Headers
 - Status codes
 - URLs
 - Request body
-

2. What is a Request?

A **request** is a message sent by the client to the server.

Example:

GET /api/v1/employees/10 HTTP/1.1

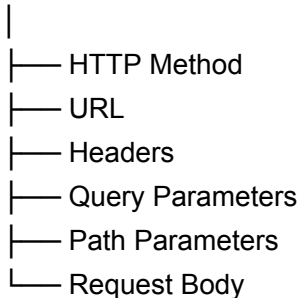
Host: api.example.com

Authorization: Bearer <access-token>

Accept: application/json

A request can contain:

REQUEST



Not every request needs every part.

3. What is a Response?

A **response** is the message returned by the server.

Example:

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "success": true,
  "data": {
    "id": 10,
    "name": "Sagar Maurya",
    "email": "sagar@example.com"
  }
}
```

```
}
```

A response normally contains:

RESPONSE

```
|  
├── Status Code  
├── Headers  
└── Body
```

4. What is JSON?

JSON (JavaScript Object Notation) is one of the most commonly used formats for exchanging data in REST APIs.

Example:

```
{  
  "firstName": "Sagar",  
  "lastName": "Maurya",  
  "email": "sagar@example.com"  
}
```

JSON is popular because it is:

- Lightweight
- Easy to read
- Easy to generate and parse
- Supported by most programming languages

A client can send JSON to the API, and the API can return JSON.

5. What are Headers?

Headers contain additional information about a request or response.

Example:

Content-Type: application/json

Authorization: Bearer <token>

Accept: application/json

Content-Type

Tells the server what format the request body uses.

Content-Type: application/json

Authorization

Carries authentication credentials such as an access token.

Authorization: Bearer <access-token>

Accept

Tells the server which response format the client prefers.

Accept: application/json

Other common headers include:

User-Agent

Cache-Control

Content-Length

Origin

6. What is a URL?

A URL identifies an API resource.

Example:

`https://api.example.com/api/v1/employees/10`

Breakdown:

`https://`

|
└─ Protocol

api.example.com

|
└─ Domain / Server

/api

|
└─ API prefix

/v1

|
└─ API version

/employees

|
└─ Resource

/10

|
└─ Specific resource

Good REST APIs generally use resource-oriented URLs.

Example:

/api/v1/employees

/api/v1/departments

/api/v1/leave-requests

Instead of action-oriented URLs such as:

/api/getAllEmployees

/api/createEmployee

/api/deleteEmployee

The HTTP method already describes the intended operation.

7. Query Parameters

Query parameters are commonly used for:

- Filtering
- Searching
- Sorting
- Pagination
- Optional configuration

Example:

GET /api/v1/employees?department=2

Here:

department=2

is a query parameter.

Multiple parameters:

GET /api/v1/employees?department=2&status=active

Pagination

GET /api/v1/employees?page=2&limit=20

Searching

GET /api/v1/employees?search=sagar

Sorting

GET /api/v1/employees?sortBy=createdAt&order=desc

A real API can combine them:

GET /api/v1/employees
?search=sagar
&department=2
&page=1
&limit=20

&sortBy=createdAt
&order=desc

8. Path Parameters

Path parameters are used to identify a specific resource.

Example:

GET /api/v1/employees/15

Here:

15

is the path parameter.

The idea is:

/employees/15
↑
Specific employee

Compare it with:

/employees?department=2
↑
Filter employees

Path parameter

Used to identify a particular resource:

/employees/15

Query parameter

Used to filter or modify how resources are returned:

9. Request Body

The request body contains data sent by the client.

Example:

POST /api/v1/employees

Content-Type: application/json

```
{  
  "firstName": "Sagar",  
  "lastName": "Maurya",  
  "email": "sagar@example.com",  
  "departmentId": 2  
}
```

The request body is commonly used with:

POST

PUT

PATCH

GET and DELETE requests generally do not require a body for ordinary REST designs.

10. HTTP Methods

The most important HTTP methods are:

Method	Common purpose
GET	Read data
POST	Create a resource or perform a non-idempotent operation
PUT	Replace a resource representation

PATCH Partially modify a resource

DELETE Delete a resource

11. GET

GET is used to retrieve data.

Get all employees

GET /api/v1/employees

Get one employee

GET /api/v1/employees/15

Example response:

```
{
  "success": true,
  "data": {
    "id": 15,
    "firstName": "Sagar",
    "email": "sagar@example.com"
  }
}
```

GET should normally be used for operations that do not modify server state.

12. POST

POST is commonly used to create a new resource.

Example:

POST /api/v1/employees

Content-Type: application/json

```
{
  "firstName": "Sagar",
  "lastName": "Maurya",
  "email": "sagar@example.com",
  "departmentId": 2
}
```

The server may return:

201 Created

```
{
  "success": true,
  "message": "Employee created successfully",
  "data": {
    "id": 101,
    "firstName": "Sagar",
    "email": "sagar@example.com"
  }
}
```

Typical processing:

Request



Authentication



Validation



Business Logic



Database



Response

13. PUT

PUT is generally used to replace a resource representation.

Suppose the resource is:

```
{
  "name": "Sagar",
  "email": "sagar@example.com",
  "phone": "9876543210"
}
```

A PUT request might contain the complete representation:

PUT /api/v1/employees/101

```
{
  "name": "Sagar Maurya",
  "email": "sagar.maurya@example.com",
  "phone": "9999999999"
}
```

Conceptually:

Old Resource

↓

Replace

↓

New Resource

14. PATCH

PATCH is used for partial modification.

Example:

PATCH /api/v1/employees/101

```
{
  "email": "newemail@example.com"
}
```

Only the email is changed.

Conceptually:

Old Resource

```
|  
├── name    → unchanged  
├── email   → changed  
└── phone   → unchanged
```

15. PUT vs PATCH

Suppose the resource is:

```
{  
  "name": "Sagar",  
  "email": "sagar@example.com",  
  "phone": "9999999999"  
}
```

PUT

Generally represents the complete replacement:

PUT /employees/10

```
{  
  "name": "Sagar",  
  "email": "sagar@example.com",  
  "phone": "8888888888"  
}
```

PATCH

Represents a partial modification:

PATCH /employees/10

```
{  
  "phone": "8888888888"  
}
```

A common practical approach is:

PUT → complete replacement

PATCH → partial update

16. DELETE

DELETE is used to delete a resource.

Example:

DELETE /api/v1/employees/101

Possible response:

204 No Content

or:

200 OK

In business applications, permanent deletion is not always desirable.

A system may instead use **soft deletion**:

isDeleted = true

This can be useful where historical records, auditing, or recovery are important.

Examples:

HR systems

Financial systems

Order systems

Invoice systems

Audit systems

17. HTTP Status Codes

Understanding status codes is essential for production APIs.

2xx — Successful Requests

200 OK

Request succeeded.

GET /employees/10
→ 200

201 Created

A new resource was created.

POST /employees
→ 201

204 No Content

Operation succeeded and there is no response body.

DELETE /employees/10
→ 204

18. 4xx — Client Errors

400 Bad Request

The request is invalid.

Example:

```
{  
  "email": "abc"  
}
```

when a valid email is required.

401 Unauthorized

Authentication is missing or invalid.

GET /api/v1/profile

Authorization: missing

Response:

401 Unauthorized

Meaning:

Valid authentication is required.

403 Forbidden

The user is authenticated but does not have permission.

Example:

Employee → cannot approve HR request

HR → can approve

404 Not Found

The requested resource does not exist.

GET /employees/999999

Response:

404 Not Found

409 Conflict

The request conflicts with existing data or state.

Example:

POST /employees

email = sagar@example.com

If that email already exists:

409 Conflict

422 Unprocessable Content

The request is structurally valid, but the submitted data fails semantic validation.

Example:

age = -5

19. 5xx — Server Errors

500 Internal Server Error

An unexpected error occurred on the server.

Possible reasons:

Database failure

Unexpected application exception

Third-party service failure

Infrastructure problem

Do not expose internal implementation details to the client.

Bad:

```
{
  "error": "DatabaseError: connection failed at..."
}
```

Better:

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
```

```
"message": "Something went wrong"
}
```

Detailed technical information should be recorded in server logs.

20. REST API Is More Than CRUD

REST is not simply:

REST = CRUD

CRUD is just one common API design pattern.

Real business applications often contain workflows.

For example, a leave-management system could have:

POST /api/v1/leave-requests

Then:

POST /api/v1/leave-requests/501/manager-approval

Then:

POST /api/v1/leave-requests/501/hr-approval

Then:

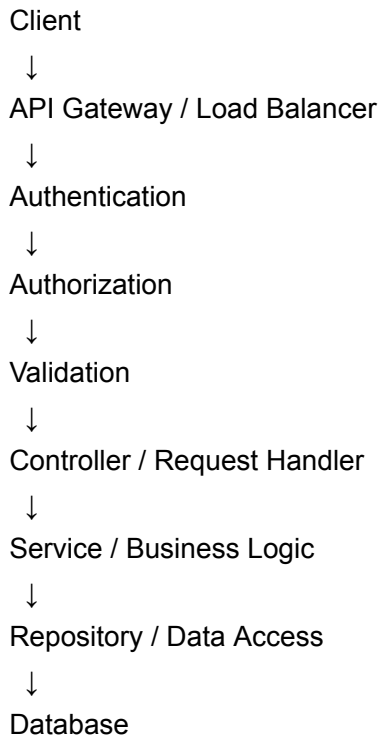
POST /api/v1/leave-requests/501/cancel

The API is representing the business domain and its state transitions.

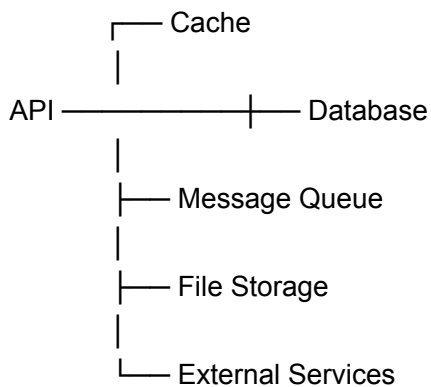
21. Production API Architecture

A production backend usually separates responsibilities.

A common logical structure is:



Additional systems may exist:



22. Authentication

Authentication answers:

Who is the user?

For example:

Authorization: Bearer <access-token>

The server verifies the credentials/token and identifies the user.

Common approaches include:

Session-based authentication

JWT

OAuth 2.0

OpenID Connect

23. Authorization

Authorization answers:

What is this user allowed to do?

Example roles:

Employee

Manager

HR

Admin

Possible permissions:

Employee

→ view own profile

→ submit leave

Manager

→ view team members

→ approve team leave

HR

→ manage employees

→ approve HR requests

Admin

→ manage system settings

Therefore:

Authentication



Who are you?



Authorization



What can you do?

24. Request Validation

Never blindly trust client input.

Example invalid request:

```
{  
  "email": "hello",  
  "departmentId": "abc"  
}
```

A production API should validate:

Required fields

Data types

String length

Email format

Number ranges

Allowed values

Date ranges

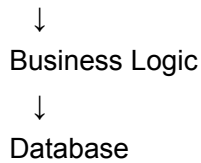
Business rules

Flow:

Request



Validation



25. Consistent Error Responses

Production APIs should use a consistent structure.

Example:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request data",
    "details": [
      {
        "field": "email",
        "message": "Invalid email address"
      }
    ]
  }
}
```

Another example:

```
{
  "success": false,
  "error": {
    "code": "EMPLOYEE_NOT_FOUND",
    "message": "Employee not found"
  }
}
```

Consistency makes frontend and mobile development much easier.

26. API Versioning

A production API may use versioning:

`/api/v1/employees`

Later:

`/api/v2/employees`

Versioning helps when introducing breaking changes.

For example:

v1 → old clients

v2 → new clients

Clients can migrate without the server immediately breaking all existing integrations.

27. Pagination

Never return millions of records in a single response.

Instead:

GET `/api/v1/employees?page=1&limit=20`

Example:

```
{
  "success": true,
  "data": [
    {},
    {},
    {}
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
```

```
"total": 350,  
"totalPages": 18  
}  
}
```

For extremely large datasets, cursor-based pagination can be more efficient than traditional page/offset pagination.

28. Filtering

Example:

GET /api/v1/employees?departmentId=2&status=ACTIVE

The API returns only matching records.

Filtering is useful for:

Department

Status

Date

Location

Role

Category

Price

Availability

29. Searching

Example:

GET /api/v1/employees?search=sagar

The API may search:

First name

Last name

Email
Employee code

For large datasets, use appropriate database indexes or dedicated search systems rather than scanning the entire dataset.

30. Sorting

Example:

GET /api/v1/employees?sortBy=createdAt&order=desc

Other examples:

sortBy=name
sortBy=salary
sortBy=joiningDate

Always validate allowed sort fields to avoid unsafe or inefficient queries.

31. Rate Limiting

Production APIs should protect against excessive requests.

For example:

100 requests / minute / user

Possible uses:

Login API
Password reset
OTP API
Search API
Public APIs
Payment endpoints

Rate limiting helps reduce:

Abuse

Brute-force attempts

Resource exhaustion

Automated attacks

32. Idempotency

Idempotency is especially important for financial or repeatable operations.

Imagine:

POST /api/v1/payments

The server successfully processes the payment, but the client does not receive the response because of a network failure.

The client retries.

Without protection, the payment could be processed twice.

An idempotency key can be used:

Idempotency-Key: 8f7a9c...

The server can associate the key with the operation and prevent accidental duplicate processing.

Useful for:

Payments

Orders

Bookings

Transfers

Message sending

Other critical operations

33. Security for Production APIs

Important areas include:

HTTPS

Authentication

Authorization

Input validation

Rate limiting

CORS configuration

Secure credential storage

Password hashing

SQL injection prevention

XSS protection

CSRF protection where applicable

Secret management

Never expose:

Database passwords

Private keys

Internal stack traces

API secrets

Sensitive user information

34. Database Considerations

A production REST API must also consider the database.

Important concepts:

Transactions

Indexes

Constraints

Connection pooling

Query optimization

Pagination

Data consistency

Backups

Replication

Example:

Request



Business Logic



Transaction



Multiple Database Operations



Commit / Rollback

Transactions are especially important when one business operation changes multiple related records.

35. Logging

Production systems need structured logging.

Useful information includes:

Request ID

Timestamp

HTTP method

URL

Status code

Response time

User/service identity where appropriate

Error code

Database/query timing where appropriate

Example:

2026-09-22T12:30:20Z

GET /api/v1/employees/101

status=200

duration=42ms

requestId=abc123

Logs help diagnose production problems.

36. Monitoring

Logging tells you what happened.

Monitoring tells you how the system is behaving.

Monitor things such as:

Request rate

Error rate

Response latency

CPU usage

Memory usage

Database performance

Database connections

Queue length

Cache performance

Important metrics often include:

2xx rate

4xx rate

5xx rate

P95 latency

P99 latency

37. Health Checks

Production services commonly expose health endpoints.

Example:

GET /health

Possible response:

```
{  
  "status": "healthy"
```

}

A more detailed health check might verify:

API service

Database

Cache

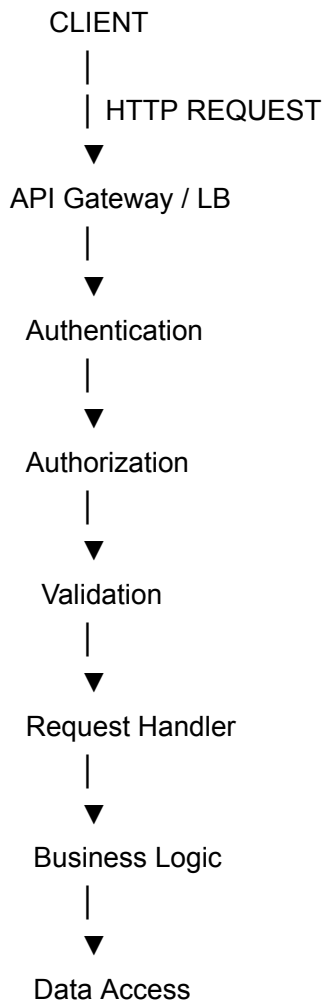
Message broker

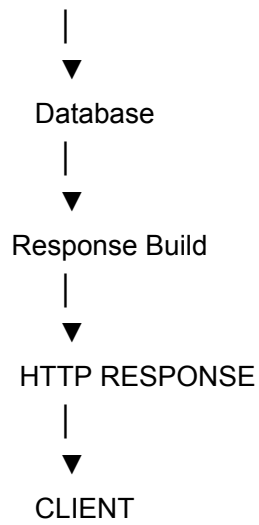
External dependencies

Health checks are useful for deployment systems and load balancers.

38. Complete REST API Request Lifecycle

Remember this overall flow:





This is the most important production-level mental model.
